UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 14

Apuntadores, Referencias y Arreglos

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Explicar qué es una dirección de memoria y cómo obtenerla con `&`.
2. Declarar apuntadores y distinguir entre el valor del apuntador y el valor apuntado.
3. Usar `*` para leer y escribir a través de un apuntador.
4. Explicar qué es una referencia y en qué se diferencia de un apuntador.
5. Aplicar aritmética de apuntadores sobre arreglos.
6. Reconocer la equivalencia `arr[i]` $\equiv$ `*(arr+i)`.
7. Identificar el acceso fuera de límites como comportamiento indefinido y sus implicaciones en seguridad.

100 minutos (sesión doble recomendada). Sugerido:
20 min — modelo de memoria y operador `&`
25 min — apuntadores: declaración, `*`, modificar valores, `nullptr`
15 min — referencias y comparación con apuntadores
25 min — arreglos, aritmética de apuntadores, equivalencia
15 min — UB, acceso fuera de límites, `vector::at()`

Prerrequisitos

- Variables, tipos de datos y operadores básicos (sesiones 1–10)
- Estructuras de control: `if`, `for`, `while` (sesiones 8–12)
- Funciones: declaración, parámetros, valor de retorno (sesiones 13–15)

Si hay alumnos con experiencia en Python, Java o JavaScript, la analogía más efectiva es: los apuntadores de C++ son como las *referencias* de esos lenguajes, pero sin gestión automática de memoria. La diferencia clave es que en C++ la desreferencia es explícita y el programador controla cuándo y cómo se accede a la memoria.

Secuencia de clase

• Apertura — El modelo de memoria (20 min)

Dibuja en el pizarrón el modelo de memoria como una tabla de celdas numeradas. Declara en vivo una variable `int x = 16;` y pregunta:

- ¿Qué necesita el compilador saber para guardar `x`? (tipo, nombre, valor)

- ¿Qué sucede en la RAM cuando se ejecuta esa línea?

Luego ejecuta en el cuaderno la primera celda y muestra la dirección hex. Haz hincapié en que la dirección cambia en cada ejecución (ASLR).

> "Si tengo `int a = 5; int b = 5;` , ¿ `&a == &b` ?"
> Respuesta: **No**. Dos variables distintas siempre tienen direcciones distintas, aunque tengan el mismo valor. El único caso en que dos nombres comparten dirección es una referencia.

• Apuntadores (25 min)

Introduce la declaración `int* p;` y enfatiza que el `*` en la declaración es parte del **tipo**, no del nombre. Muestra la distinción entre `int* p, q;` (p es apuntador, q es int) vs `int *p, *q;` .

> Los alumnos confunden el `*` de declaración con el `*` de desreferencia. Dedica tiempo explícito a esto. Un ejercicio útil: dar fragmentos de código y pedir que identifiquen el rol del `*` en cada aparición.

Demuestra el ciclo completo:

1. Declarar `int* p;`
2. Asignar `p = &x;`
3. Leer con `*p`
4. Escribir con `*p = 18;` y mostrar que `x` cambió

Introduce `nullptr` y la buena práctica de inicializar siempre los apuntadores.

• Referencias (15 min)

Muestra `int& y = x;` y pide a los alumnos que predigan qué imprimirá `&x` y `&y` antes de ejecutarlo. La mayoría esperará direcciones distintas — el resultado igual es el *momento aha* de la referencia.

Usa la tabla comparativa (apuntador vs referencia) para consolidar. Insiste en que la referencia no puede ser reasignada ni nula.

• Arreglos (25 min)

Dibuja el layout de memoria de `int arr[5] = {10,20,30,40,50};` en el pizarrón, con las direcciones. Muestra que `arr` imprime la misma dirección que `&arr[0]` .

Explica la aritmética de apuntadores con el diagrama de celdas: sumar 1 avanza `sizeof(int)` = 4 bytes. Ejecuta las celdas de `arr+1` , `arr+2` para que vean los incrementos hex.

Presenta la equivalencia `arr[i]` ≡ `*(arr+i)` y demuestra el *array decay* con `sizeof(arr)` vs `sizeof(p)` .

Muestra en vivo que `arr[2]` y `2[arr]` producen el mismo resultado. Explica que es consecuencia de la conmutatividad de la suma en la aritmética de apuntadores. Deja claro que **jamás se usa en código real** pero es útil para entender cómo funciona internamente.

- **Comportamiento indefinido y seguridad (15 min)**

Ejecuta la celda de acceso fuera de límites y muestra los resultados impredecibles. Conecta con el Gusano Morris como ejemplo histórico concreto.

Introduce `std::vector::at()` como alternativa segura. Pregunta: "¿Por qué C++ no verifica límites por defecto?" (rendimiento — dejar que el alumno razone sobre el trade-off).

Errores conceptuales frecuentes

1. "¿Por qué necesito el tipo en un apuntador si ya tengo la dirección?"
La dirección dice *dónde* está el dato, pero no *cuánto* ocupa. Sin el tipo, el compilador no sabe cuántos bytes leer al desreferenciar.

2. Confundir `p` con `*p`
`p` es la dirección. `*p` es el valor en esa dirección. Un ejercicio útil: dar `int x=5; int* p=&x;` y pedir el valor de `p`, `*p`, `&p`, `**(&p)`.

3. Creer que `int& r = x; r = y;` reasigna la referencia
No. `r = y` copia el *valor* de `y` en `x`. La referencia sigue apuntando a `x`. Las referencias no se pueden reasignar.

4. "El arreglo y el apuntador son lo mismo"
Son similares pero no idénticos. `sizeof`, no poder asignar a `arr++`, y el comportamiento con `&arr` vs `&p` los distinguen.

5. Pensar que el UB "siempre crashea"
El comportamiento indefinido puede pasar silenciosamente. Esta es la razón por la que es tan peligroso: el bug puede estar dormido y manifestarse solo en ciertas condiciones.

Conexiones con temas posteriores

- **Sesión 15** — Paso por referencia: aplica directamente apuntadores y referencias como mecanismo de paso de parámetros.
- **Sesiones 18–20** — Memoria dinámica: `new` / `delete`, que devuelven y liberan apuntadores.
- **Sesiones 21–25** — Orientación a Objetos: `this` es un apuntador al objeto actual; herencia y polimorfismo usan apuntadores a la tabla virtual (*vtable*).

- **C++ moderno** — Smart pointers (`unique_ptr`, `shared_ptr`) reemplazan los apuntadores crudos en código de producción gestionando la memoria automáticamente.

Material de la sesión

- **Cuaderno:** `sesiones/sesion14_apuntadores_referencias_arreglos.ipynb`
- **Ejemplos:** `materialInicial/sesion14/01-07_apuntadores.cpp`
- **Notas alumno:** `notas_alumno/pdf/sesion14_apuntadores_referencias_arreglos.pdf`

Rúbrica de ejercicios

Ejercicio / Criterio	Puntos	Evidencia
Ejercicio 1 — Swap con apuntadores		
Usa paso por apuntador o referencia	3	parámetros con `*` o `&`
El swap es correcto	4	valores intercambiados
No usa `std::swap`	3	implementación propia
Ejercicio 2 — Máximo con aritmética de apuntadores		
Usa `*(arr+i)` sin corchetes	4	código sin `[]`
Resultado correcto	4	máximo identificado
Recorre exactamente `n` elementos	2	sin UB
Ejercicio 3 — Conversión de temperatura con referencia		
Usa referencia en el parámetro	4	`double&` o `double*`
Fórmula correcta	4	$(F - 32) \times 5/9$
Modifica la variable original	2	sin copia
Total	**30**	